Common C++ Linker Errors Explained

LNK1169 / LNK2005 — Multiply Defined Symbols

Error Message

```
LNK2005: '...' already defined in ...
LNK1120: 1 unresolved externals (often summary)
```

Cause

The same symbol (function/variable) has two definitions across object files.

Example

```cpp
// main.cpp
int main() { ... }

// test.cpp
int main() { ... }    // duplicate!
```

Both `.obj` files end up with `main` — linker fails.

Fix

- Remove the duplicate definition
- Or exclude one file from build (right-click → Properties → Excluded From Build)
- Or mark file-local helpers as `static` or put them in anonymous namespace

LNK2019 / LNK1120 — Unresolved External Symbol

Cause

A function was DECLARED but not DEFINED, OR declaration/definition disagree on linkage.

Example 1 — Missing definition

```cpp
// header.h
void foo();

// main.cpp
foo();   // calls foo
```

```
// No source file defines foo() -> LNK2019
```

Example 2 — `extern "C"` mismatch

```
// header.h
extern "C" void PrintA(int);    // declared with C linkage (unmangled)

// source.cpp
void PrintA(int a) {}           // defined WITHOUT extern "C" (mangled)
                                // -> LNK2019/LNK1120
```

Fix

- Provide the definition
- Make declaration and definition agree on `extern "C"`

C4273 — Inconsistent DLL Linkage

Cause

You re-declared a standard library function whose original declaration uses `__declspec(dllimport)`. Your re-declaration omits it → mismatch.

Example

```
#include <math.h>   // already declares cbrt with __declspec(dllimport)

extern "C" {
    double cbrt(double x);    // your declaration - no dllimport - WARNING
}
```

Why Only Some Functions Warn

- Classic functions (`pow`, `sqrt`, `sin`, `cos`, `tan`) are MSVC **intrinsics** → re-declaration tolerated
- C99 additions (`cbrt`, `hypot`, `fmax`, `fmin`) are dllimport-only → conflict shows

Fix

- Don't re-declare standard library functions
- Just `#include <math.h>` and use them

C2169 — Intrinsic Function Cannot Be Defined

Cause

You tried to define your own version of a function that the compiler treats as an intrinsic. Triggered in Release mode (`/O2` enables `/Oi`).

Example

```
extern "C" double sqrt(double x) {
    return x + x;   // ERROR C2169 in Release: 'sqrt' is intrinsic
}
```

Why

With intrinsics ON, the compiler emits the CPU's native `sqrtsd` instruction directly inline — there's no function call to redirect. Defining a body is meaningless.

Why Debug Worked

Debug mode (`/Od`) disables intrinsics. `sqrt` is treated as a normal external function → your override succeeds.

Fix

- Rename your function (e.g., `mySqrt`)
- Never reuse standard library names for your own implementations

---

Linker Search Order — The Big Picture

```
1. FIRST: Linker processes all .obj files
   - All object file symbols are mandatory
   - Defined here → SATISFIED

2. THEN: Linker processes .lib libraries
   - Only pulls objects from a library to resolve STILL-MISSING symbols
   - Already satisfied → library version NEVER loaded
```

Why You Can "Override" Library Functions

```
// myFile.cpp
extern "C" double sqrt(double x) { return x + x; }

// main.cpp
sqrt(2.3);  // calls YOUR version (4.6), not math.h's
```

Your `sqrt` is in `myFile.obj` (always linked). It satisfies the `sqrt` symbol first. The CRT library's `sqrt` is never pulled in.

Why You DON'T Get LNK2005 Here

- `.obj` + `.obj` with same symbol → BOTH forced in → DUPLICATE → LNK2005
- `.obj` + `.lib` with same symbol → obj wins, lib skipped → NO DUPLICATE

C4172 — Returning Address of Local Variable

Warning (not error)

```
int& getDangling() {
    int local = 99;
    return local;       // C4172 warning
}
```

Why It Matters

Returns a dangling reference. Reading it is undefined behavior. **Don't ignore this warning.**

Fix

- Return by value
- Return a heap-allocated object (preferably smart pointer)
- Use `static` local if appropriate

Quick Diagnosis Table

Error	What Happened	First Thing to Check
LNK1169	Two object files define same symbol	Duplicate function definitions across .cpp files
LNK2005	Same as above (specific message)	Same as above
LNK2019	Symbol used but not defined	Missing implementation OR extern "C" mismatch
LNK1120	Linker summary of unresolved externs	Look for the LNK2019 above it
C4273	Re-declared standard fn with wrong linkage	Remove the re-declaration
C2169	Defined an intrinsic function (Release)	Rename your function
C4172	Returned ref/ptr to local	Return by value or use heap

Symbol Mangling Quick Reference (MSVC)

```
void foo()              ->  ?foo@@YAXXZ
void foo(int)           ->  ?foo@@YAXH@Z
int foo(int)            ->  ?foo@@YAHH@Z
```

```
void foo(int, double)  ->  ?foo@@YAXHN@Z
extern "C" void foo()  ->  foo                  (no mangling)
```

Code letters:

- `H` = int, `N` = double, `M` = float, `D` = char
- `X` = void, `_N` = bool

Golden Rules

```
1. Header declares, source defines.
2. Declaration and definition must AGREE on extern "C", linkage, signature.
3. Don't re-declare standard library functions.
4. Don't reuse names of compiler intrinsics (sqrt, pow, etc.).
5. Object files (.obj) ALWAYS link. Libraries (.lib) only when needed.
6. Use anonymous namespace or static for file-private helpers.
7. Never ignore warnings - especially C4172, C4273.
```